

Edge-weight semantics + unnormalized HHI — final report

AI Supply Chain Terminal

2026-07-21

1. Edge-weight semantics audit — what the weights mean today

Across all five in-edge types, per-target sums by edge type:

edge type	targets	targets with sum > 1.0	min	median	p90	max
mines	5	1	0.65	1.00	1.17	1.17
refines	5	2	0.90	1.00	1.10	1.10
supplies	23	9	0.30	1.00	1.70	1.75
input_to	21	3	0.08	0.25	1.45	1.85
operates	5	0	1.00	1.00	1.00	1.00

The intended semantics for HHI to work as “supplier-share concentration” is *this source’s share of this target’s supply*. That reading requires per-target sums to be at most 1.0.

Findings:

- **mines / refines** are close to the intended reading. Most targets sum ≤ 1.0 . A few slightly over (neodymium mines 1.17, refines 1.10) — modelling both countries and facilities as separate mining sources double-counts Mountain Pass under both USA and its own facility node. Not a semantic mismatch, a graph- double-count issue.
- **supplies** systematically does NOT reflect target-input-share. 9 of 23 targets have inbound `supplies` weights summing above 1.0; TSMC and Meta both at 1.75. Reading: current `supplies` weights are authored as “this supplier’s share of ITS OWN output going to this target” (or “this supplier’s importance to this target’s stage”) — not “target’s input from this source”. Two different quantities live behind one field.
- **input_to** is internally inconsistent. Some targets sum tiny (TSMC 0.08 = copper only; several data-centre facilities 0.08 = NdFeB only), others sum above 1.0 (NVIDIA 1.85 = HBM 0.90 + CoWoS 0.95). The high-summing cases were authored as *criticality of dependency* — “an HBM shortage devastates NVIDIA, so the weight is 0.9” — not as input-mix share. Same field, two readings.

The intended semantics is now documented as a docstring on `Edge` in `backend/app/schema/edge.py`, and referenced from `config/scoring.yaml`. Weights have NOT been rewritten — that is its own task with its own review.

2. Implementation — `normalize` flag

`config/scoring.yaml`:

```
concentration:
  inbound:
    per_stage:
      normalize: true          # default – no scores change
```

- `compute_hhi(shares, normalize=True)` — divides by sum, then squares. Legacy.
- `compute_hhi(shares, normalize=False)` — sum of squared raw shares. Incomplete buckets self-report.

The flag threads through `compute_stage_hhis`, `per_stage_hhi`, and `hhi_from_derived_shares` via `config.inbound_per_stage_normalize`. No output clamping. When shares sum above 1.0 the unnormalized HHI can exceed 1.0 — the semantic mismatch surfaces rather than being hidden.

3. Assertions

#	assertion	outcome
1	<code>normalize=true</code> reproduces current scores (default was true; verified)	OK
2	Gallium <code>mined_by_hhi</code> identical under both modes (0.9704)	OK
3	TSMC <code>input_to</code> HHI unnormalized = 0.00640, < 0.05	OK
4a	No HHI outside [0, 1] under normalized	OK
4b	No HHI outside [0, 1] under un normalized	FAIL — 3 nodes hold <code>inbound_hhi > 1.0</code> because their <code>supplies</code> bucket sums above 1.0. A finding, not a bug in the measure. Reflects the semantics mismatch in §1. Nodes: TSMC (1.003), NVIDIA (1.712), Meta (1.132), AMD (1.285), Google (1.150).
5	All 63 nodes score and tier in both modes	OK
6	Monotonicity — unnormalized HHI non-decreasing under positive addition	OK

4. Comparison — the answers

3a. Paper chokepoints under each mode

node	normalized	unnormalized
TSMC	critical (0.469)	critical (0.470)
ASML	critical (0.393)	critical (0.393)
gallium	critical (0.480)	critical (0.480)
dysprosium	critical (0.545)	critical (0.556)
HBM	critical (0.234)	critical (0.234)
CoWoS	critical (0.313)	critical (0.313)
RF & Power Semis	critical (0.319)	moderate (0.115)

Six of seven hold. **RF & Power drops from critical to moderate** — the case the spec called out as decisive.

3b. RF & Power specifically

- `input_to` bucket: a single edge from gallium at weight 0.60.
- `input_to` HHI: 1.000 normalized $\rightarrow$ 0.360 unnormalized ($0.60^2 = 0.36$).
- `inbound_hhi`: 1.000 $\rightarrow$ 0.360.
- `concentration`: 1.000 $\rightarrow$ 0.360.
- `severity`: 0.319 $\rightarrow$ 0.115.
- `tier`: **critical** $\rightarrow$ **moderate**.

The 0.60 weight for gallium $\rightarrow$ RF & Power was authored as “gallium is 60% of RF & Power’s material input.” Under unnormalized, 0.60 IS the concentration read; 0.36 falls into “supply is concentrated among a few sources” rather than “one source controls almost all supply.”

Two readings that are both defensible are open for review:

- The measure is wrong for the RF & Power case — 60% single-supplier reading should count as critical since there is no other supplier at all.
- The 0.60 weight understates gallium’s actual role in RF/Power semiconductors — real GaAs/GaN devices are ~100% gallium-based, so the *data* is the wrong side of the trade-off, not the measure.

No adjustments made. Reported as the open question the spec anticipated.

3c. TSMC dominant_axis

The spec anticipated the axis would return to `outbound` under unnormalized — this did **not** happen.

- normalized: inbound 1.000 · outbound 1.000 · dominant_axis = `inbound` (tie-break to inbound)
- unnormalized: inbound **1.003** · outbound 1.000 · dominant_axis = `inbound`

TSMC's `supplies` bucket sums to 1.75 (ASML 0.90 + AMAT 0.30 + LRCX 0.20 + KLA 0.15 + TEL 0.20). Under unnormalized: $0.81 + 0.09 + 0.04 + 0.0225 + 0.04 = 1.0025$, above 1.0. The bucket dominates outbound (1.000) by 0.003, so `inbound` still wins the tie.

This is the semantics mismatch from §1 surfacing directly. If `supplies` weights were rewritten to $\text{sum} \leq 1.0$ per target (target-input-share reading), TSMC's `supplied_by_hhi` unnormalized would be ~0.35 and the axis would flip to outbound as expected.

3d. Movement summary

16 nodes shift tier under unnormalized. Split:

- 2 up: Neodymium high → critical; AMD, NVIDIA moderate → high (via `input_to` buckets that exceed 1.0 unnormalized — same semantic mismatch)
- 1 sideways-down that matters most: RF & Power critical → moderate
- 13 down: SK Hynix, Micron, Siemens Energy, GE Vernova, Samsung Electronics, Quanta, Vertiv, Colossus, Stargate, Vantage, Citadel, Constellation Energy, Samsung.

Correlation with the share-completeness backlog: 13 of 16 shifting nodes are in the backlog.

This is the direct evidence the spec asked for: incomplete nodes are the ones that move, in the direction the measure should move them (down toward "low concentration reading from thin data"). The three that go up are the ones the semantics mismatch touches (`supplies` and high-weight `input_to` sums > 1.0).

5. TSMC + RF & Power narrations under both modes

TSMC — normalized (current) · dominant_axis=inbound · critical > One source controls almost all supply (1.00 on a 0–1 scale), substitutes are > marginal, and replacing it would take roughly three to five years.

TSMC — unnormalized · dominant_axis=inbound · critical > One source controls almost all supply (1.00 on a 0–1 scale), substitutes are > marginal, and replacing it would take roughly three to five years.

(Reads identically because TSMC's inbound_hhi lands ~1.00 under both — the supplies-sum-above-1.0 issue described in §3c.)

RF & Power Semiconductors — normalized (current) · critical > One source controls almost all supply (1.00 on a 0–1 scale), limited > substitutes exist, and replacing it would take roughly three to five years.

RF & Power Semiconductors — unnormalized · moderate > Supply is concentrated among a few sources (0.36 on a 0–1 scale), limited > substitutes exist, and replacing it would take roughly three to five years.

RF & Power is the node the previous correction existed to restore. Under unnormalized it does not hold — see §3b for the framing.

6. Test suite — both modes

Normalized (current default): 1 failed, 24 passed.

- share-completeness fails (pre-existing, per the test suite spec).

Unnormalized: 3 failed, 22 passed.

- share-completeness — same pre-existing failure, unaffected by mode (reads raw sums, not HHI, as the spec noted).
- **bounds** — 5 nodes hold `inbound_hhi > 1.0` under unnormalized. Same semantics-mismatch as §3c.
- **paper chokepoints** — RF & Power drops out of critical. Same as §3a/b.

Monotonicity test (new, `test_hhi_monotonicity.py`, three cases): **passes**. The unnormalized measure is non-decreasing under positive addition, and matches normalized exactly when a bucket sums to 1.0 (gallium's mines).

7. Recommendation

Do not switch the default to `normalize: false` in this task. Reasoning:

1. The direct comparison is currently confounded by the edge-weight semantics mismatch in §1. Under a mixed data model — where `mines / refines` are target-input-share, `supplies` is source-output-share, `input_to` is a mix of both — neither `normalize: true` nor `normalize: false` gives an internally consistent answer.
2. `normalize: true` accidentally papers over the mismatch by rescaling every bucket to sum 1.0. That is doing the wrong thing for the right-feeling result: readings are stable but incompleteness is invisible. This is the root cause of the thin-graph problem and it will continue to bite as more nodes are added.
3. `normalize: false` shows the mismatch (bounds failure) and shows the thin-graph problem (13 of 16 tier drops correlate with the backlog). It does not yet produce trustworthy scores, because the audit's semantic questions have not been resolved.

Concrete next step, in order:

- a. Reconcile weight semantics for `supplies` and `input_to`: pick "share of target's input" as the single reading, and rewrite existing edges to satisfy it. Weights above 1.0 become editorial errors, not free parameters. This is its own task with its own review — the spec was right to keep it out of scope here.
- b. Once (a) is done, unnormalized becomes the honest measure — incompleteness dampens, completeness reveals concentration. `normalize: false` becomes the new default at that point.
- c. Threshold recalibration follows (a) and (b), not before.

In the meantime, `normalize: false` is available behind the flag for inspection. The flag is the audit tool while the data is being reconciled.

8. Files changed

backend/app/schema/edge.py	– weight semantics documented on Edge
backend/app/scoring/engine.py	– compute_hhi(normalize=...), threaded through per_stage / combined
backend/app/scoring/config.py	– inbound_per_stage_normalize accessor
config/scoring.yaml	– normalize: true (default)
backend/tests/fixtures/scoring.yaml	– fixture synced
backend/tests/test_hhi_monotonicity.py	– new test (3 cases, all pass)

No edge additions, no weight edits, no thresholds moved. Cascade, outbound criticality, rating, narration code, `narration.yaml`, and the frontend are untouched.